

Práctica 1: Aprendizaje Supervisado

Comparativa de Algoritmos de Clasificación sobre el Dataset Titanic

Mauro Valls Vidal
Universidad de Alicante

Resumen

En este trabajo se aplican los principales algoritmos de aprendizaje supervisado sobre el dataset del Titanic, incluyendo Support Vector Machines (SVM), Árboles de Decisión, Random Forest, Gradient Boosting y AdaBoost. Se compara el rendimiento de cada algoritmo, se analizan los efectos de sus hiperparámetros y se selecciona el mejor modelo global. El mejor resultado se obtiene con SVM (kernel RBF, $C=1$, $\gamma=0.1$), alcanzando un 83.85 % de accuracy en validación cruzada y un 80.45 % en test.

1. Preprocesamiento y Pipeline de Datos

Transformamos los datos brutos en matrices numéricas limpias para evitar fugas de información. Todo el preprocesamiento se realizó en la Práctica 0 y aquí se reutiliza directamente.

1.1. Ingeniería y Limpieza

- **FamilySize.** Agrupamos `SibSp` y `Parch` en una sola métrica de contexto familiar para simplificar el modelo. Se calcula como `SibSp + Parch + 1`.
- **Eliminación de ruido.** Descartamos columnas como `Name`, `Ticket`, `Cabin` y `PassengerId` porque su alta cardinalidad aporta ruido y riesgo de sobreajuste.

1.2. Pipeline de Transformación

Usamos un `ColumnTransformer` para automatizar el tratamiento según la naturaleza del dato. Las variables numéricas (`Age`, `Fare`, `SibSp`, `Parch`, `Pclass` y `FamilySize`) se imputan con la mediana y se escalan con `StandardScaler`. Las variables categóricas (`Sex` y `Embarked`) se imputan con la moda y se codifican mediante `OneHotEncoder`.

Cuadro 1: Estrategia de preprocesamiento por tipo de dato

Tipo	Tratamiento	Justificación
Numérico	<code>SimpleImputer</code> (mediana) + <code>StandardScaler</code>	Robustez frente a valores extremos
Catégorico	<code>SimpleImputer</code> (moda) + <code>OneHotEncoder</code>	Convierte texto en binarios

1.3. Partición Estratificada

Dividimos los datos en 80/20 usando `stratify=y` para mantener la proporción real de supervivientes en ambos conjuntos y asegurar una evaluación objetiva. Esto resulta en **712 muestras de entrenamiento** y **179 muestras de test**, ambos conjuntos completamente numéricos y sin valores nulos.

2. Parte 1: Support Vector Machines

2.1. SVM con Diferentes Kernels

Entrenamos cuatro configuraciones de Máquinas de Vector de Soporte para comparar cómo afectan los diferentes kernels a la frontera de decisión.

2.1.1. Escalado Obligatorio

Las SVM basan su cálculo en distancias por lo que aplicamos un `StandardScaler` dentro de un `Pipeline`. Esto garantiza que todas las variables tengan el mismo peso y evita que el modelo ignore características con rangos numéricos menores.

2.1.2. Configuración de Kernels

Probamos cuatro aproximaciones distintas para proyectar los datos y encontrar el hiperplano óptimo.

- **Linear.** Busca una separación lineal directa entre clases.
- **Poly (grado 3).** Crea una frontera polinómica para capturar relaciones no lineales.
- **RBF.** Utiliza funciones de base radial para manejar distribuciones circulares o complejas.
- **Sigmoid.** Aplica una transformación similar a las funciones de activación de redes neuronales.

Usamos Validación Cruzada con 5 particiones para asegurar la estabilidad del modelo. Monitorizamos el número de Vectores de Soporte (SV) porque un porcentaje demasiado alto respecto al total de entrenamiento suele ser un indicador de sobreajuste.

Cuadro 2: Comparación de kernels SVM con hiperparámetros por defecto

Kernel	CV Acc	Test Acc	Prec.	Recall	F1	N° SV	% Train
Linear	0.7894	0.7765	0.7377	0.6522	0.6923	332	46.6 %
Poly (d=3)	0.8287	0.8045	0.8696	0.5797	0.6957	330	46.3 %
RBF	0.8385	0.8045	0.8400	0.6087	0.7059	326	45.8 %
Sigmoid	0.6939	0.6704	0.5926	0.4638	0.5203	295	41.4 %

2.1.3. Análisis de Resultados

Comparamos el desempeño de los modelos para identificar cuál captura mejor los patrones de supervivencia del Titanic.

El Ganador Es RBF. El kernel RBF obtiene la mejor precisión tanto en validación cruzada (83.85 %) como en test (80.45 %). Es el modelo más equilibrado con el F1-Score más alto (0.7059). Su flexibilidad matemática le permite adaptarse a la frontera de decisión compleja de este problema sin caer en un sobreajuste excesivo.

El Comportamiento del Poly ($d=3$). Aunque empata en precisión total con RBF muestra un sesgo interesante.

- **Alta Precisión (86.96 %).** Cuando el modelo predice que alguien sobrevive casi siempre acierta.
- **Bajo Recall (57.97 %).** Se le escapan muchos supervivientes reales (falsos negativos).

Es un modelo conservador que solo se arriesga a predecir “sobrevive” cuando la evidencia es muy clara.

Rendimiento y Complejidad. El kernel Sigmoid es el menos eficiente con una caída drástica en todas las métricas porque su transformación no encaja con la estructura lógica de los datos del Titanic. El uso del 46 % de los datos como soporte confirma que las clases están muy solapadas, obligando al modelo a emplear casi la mitad del dataset para definir una frontera de decisión fiable.

Conclusión. El kernel RBF es la opción más robusta porque ofrece la mejor capacidad de generalización y el mejor equilibrio entre precisión y recuperación.

2.2. Optimización del Kernel RBF

Afinamos el modelo mediante `GridSearchCV` para encontrar el equilibrio óptimo entre sesgo y varianza explorando el espacio de hiperparámetros.

Cuadro 3: Papel de los hiperparámetros del kernel RBF

Parámetro	Función	Efecto
C	Regularización	Controla la penalización por errores. C alto $\rightarrow$ overfitting
gamma	Influencia	Define el radio de acción. gamma alto $\rightarrow$ frontera compleja

Cuadro 4: GridSearch RBF: combinaciones representativas (ordenadas por CV Accuracy)

C	γ	CV Acc	Test Acc	N° SV	% Train	t (s)
1	0.1	0.8385	0.8045	326	45.8 %	0.014
1	scale	0.8385	0.8045	326	45.8 %	0.012
1	auto	0.8385	0.8045	326	45.8 %	0.016
100	0.01	0.8315	0.8101	285	40.0 %	0.024
10	auto	0.8217	0.8156	305	42.8 %	0.016
10	0.1	0.8217	0.8156	305	42.8 %	0.016
10	0.01	0.8161	0.8212	302	42.4 %	0.010
1	1	0.8034	0.7933	441	61.9 %	0.020
100	0.1	0.8020	0.7709	294	41.3 %	0.029
10	1	0.8006	0.7877	421	59.1 %	0.022
100	0.001	0.7964	0.7877	314	44.1 %	0.014

2.2.1. Análisis de la Optimización RBF

Los resultados muestran que **C=1** y **gamma=0.1** son los valores óptimos. El hecho de que **scale** y **auto** obtengan métricas idénticas confirma que el preprocesamiento de las variables ha sido efectivo y que la escala de los datos es uniforme.

Efecto de C. Al subir a C=100 el acierto en test cae drásticamente a 0.7709. El modelo se vuelve demasiado estricto con los errores de entrenamiento y pierde flexibilidad frente a datos nuevos.

Efecto de gamma. Con gamma=1 el número de vectores de soporte sube al 61.9 % del dataset. Esto indica una frontera de decisión demasiado sinuosa que intenta rodear puntos individuales en lugar de capturar la tendencia general.

El modelo seleccionado ofrece un Test Accuracy de 0.8045. Aunque existen combinaciones con un acierto puntual en test algo mayor (como C=10 y gamma=0.01), mantenemos la ganadora de la validación cruzada porque es la más consistente estadísticamente.

Conclusión. La configuración elegida garantiza un modelo equilibrado que utiliza el 45.8 % de los datos como soporte para definir una frontera robusta y eficiente.

2.3. Optimización del Kernel Polinomial

Optimizamos un kernel polinomial que nos permite encontrar fronteras de decisión no lineales al combinar las características existentes. Usamos **GridSearchCV** para evaluar 27 combinaciones distintas de hiperparámetros (con 5 iteraciones de validación cruzada cada una).

- **degree** (2, 3 y 4). Controla la complejidad de la frontera.
- **C** (0.1, 1 y 10). Regula la penalización de los errores.
- **gamma** (**scale**, **auto**, 0.1). Determina la escala del kernel.

Cuadro 5: GridSearch Polinomial: mejores configuraciones

degree	C	γ	CV Acc	Test Acc	N° SV
3	1	auto	0.8287	0.8045	330
3	10	0.1	0.8217	0.7821	288
2	1	0.1	0.8119	0.8045	388
4	10	auto	0.8105	0.7933	314
2	10	scale	0.8105	0.7821	291
4	1	scale	0.7978	0.7933	386
3	0.1	0.1	0.7739	0.7598	481

2.3.1. Análisis de Resultados

El grado 3 es el punto óptimo. Este valor ha demostrado ser el factor más determinante con una precisión en validación cruzada de 0.8287 y de 0.8045 en test. El grado 2 sufre un ajuste insuficiente mientras que el grado 4 empieza a perder rendimiento por sobreajuste. En el contexto del Titanic las relaciones de tercer orden son suficientes para trazar una frontera no lineal efectiva.

Impacto de C. El valor 1.0 ofrece el mejor equilibrio. Si bajamos a 0.1 la precisión cae drásticamente y el número de vectores de soporte se dispara hasta 538 porque el margen es tan permisivo que casi cualquier punto termina influyendo en la frontera. Por otro lado subir a 10 no aporta mejoras y crea una frontera demasiado estricta.

La escala de gamma no influye. No se ha observado ninguna alteración en los resultados al cambiar entre `auto`, `scale` o 0.1. Las métricas se mantienen fijas en 330 vectores de soporte para las mejores combinaciones.

Conclusión. La configuración de grado 3 y $C=1.0$ es la más potente para este kernel, aunque no logra superar al kernel RBF.

2.4. Comparación de Implementaciones (Kernel Lineal)

Ponemos a prueba tres versiones diferentes de un clasificador lineal para ver cuál se comporta mejor. Probamos el SVC convencional con kernel lineal, la implementación optimizada de `LinearSVC` y el `SGDClassifier` configurado con pérdida hinge. Aunque en teoría los tres deberían dar resultados parecidos, sus algoritmos internos de optimización son distintos y eso influye tanto en la velocidad como en la precisión final.

Cuadro 6: Comparación de implementaciones lineales

Implementación	CV Acc	Test Acc	N° SV	Tiempo (s)
SVC(linear)	0.7894	0.7765	332	0.012
LinearSVC	0.7936	0.7933	N/A	0.003
SGDClassifier	0.7781	0.7095	N/A	0.000

2.4.1. Análisis de Resultados

LinearSVC es la mejor opción. Ha sacado la precisión más alta con un 0.7936 en validación y un 0.7933 en el test. Además de ser el más equilibrado es mucho más rápido que el SVC normal al estar optimizado específicamente para casos lineales.

El **SGDClassifier** es increíblemente rápido pero no merece la pena. Su precisión en el test baja hasta el 0.7095 y eso indica que es demasiado inestable para este dataset. La brecha tan grande entre la validación y el test sugiere que el algoritmo no ha llegado a converger correctamente.

Conclusión. El ganador indiscutible es **LinearSVC** por su gran capacidad de generalización y su rapidez.

2.5. Análisis de Vectores de Soporte

Comprobamos cuántos puntos del dataset está usando el modelo de verdad para marcar el límite entre las clases. Saber qué porcentaje del entrenamiento representan es fundamental porque nos indica si el modelo es demasiado complejo o si ha logrado un buen equilibrio.

Cuadro 7: Vectores de soporte por configuración

Configuración	N° SV	% Training Set
Linear (C=1)	332	46.63 %
RBF (C=1, γ =scale)	326	45.79 %
RBF (C=10, γ =0.1)	305	42.84 %
RBF (C=100, γ =1)	411	57.72 %
Poly (d=3, C=1)	330	46.35 %

2.5.1. Análisis de Resultados

Los modelos Linear, RBF (C=1) y Poly se mantienen cerca del 46 % de vectores de soporte, indicando que el modelo necesita una cantidad considerable de puntos para separar las clases pero sin saturar el algoritmo.

La mejor opción en cuanto a limpieza es el **RBF con C=10 y γ =0.1**. Solo necesita un 42.84 % de los datos para definir la frontera, convirtiéndolo en el modelo más eficiente.

El caso del **RBF con C=100 y γ =1** es el más problemático porque el número de vectores se dispara hasta el 57.72 %. Superar la mitad del dataset para marcar el límite prueba que el modelo está memorizando el ruido. Esta configuración tiene más riesgo de sobreajuste.

Conclusión. Lo normal para este problema es moverse en torno al 45 % de vectores de soporte. Debemos evitar los parámetros que suban este porcentaje por encima del 50 %.

2.6. Evaluación Final (Parte 1)

Analizamos a fondo el rendimiento del mejor modelo SVM. La Tabla 8 resume los mejores modelos de cada kernel y la Figura 1 muestra la matriz de confusión.

Cuadro 8: Resumen Parte 1: mejor modelo de cada kernel

Modelo	CV Acc	Test Acc	F1	N° SV	% Train
SVM Linear (mejor C)	0.7894	0.7765	0.6923	332	46.6 %
SVM Poly (mejor config)	0.8287	0.8045	0.6957	330	46.3 %
SVM RBF (mejor config)	0.8385	0.8045	0.7059	326	45.8 %
SVM Sigmoid	0.6939	0.6704	0.5203	295	41.4 %

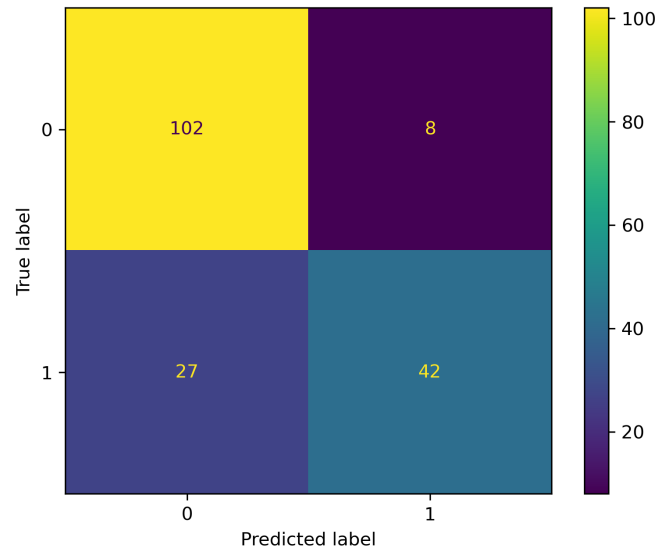


Figura 1: Matriz de confusión del mejor SVM (RBF, $C=1$, $\gamma=0.1$)

El mejor modelo alcanza un 80 % de precisión en test. El clasificador es muy fiable para predecir quién no sobrevivió (recall de 0.93) pero tiene más dificultades con los supervivientes (recall de 0.61), indicando un modelo algo conservador. El SVM RBF destaca como el ganador absoluto con el mejor F1-Score (0.7059) y la mayor consistencia en validación cruzada.

Conclusión final de la Parte 1. El kernel RBF con su configuración optimizada es la mejor herramienta para este problema. Logra captar mejor la complejidad del Titanic y mantiene un porcentaje de vectores de soporte del 45.8 % que garantiza buena generalización.

3. Parte 2: Árboles de Decisión

3.1. Árbol sin Restricciones (Baseline)

Creamos un modelo base entrenando un árbol de decisión sin aplicar ninguna limitación en su crecimiento. El objetivo es observar de forma clara el fenómeno del sobreajuste.

Cuadro 9: Árbol de decisión sin restricciones

Configuración	Train Acc	Test Acc	Profundidad	N° Hojas
Sin restricciones	0.9831	0.8045	20	159

Existe una brecha enorme entre la precisión de entrenamiento de 0.9831 y la de test que se queda en 0.8045. Esta diferencia es el ejemplo perfecto de overfitting porque el modelo ha memorizado prácticamente todos los detalles del entrenamiento pero no es capaz de aplicar ese conocimiento con la misma eficacia en datos nuevos. La complejidad es excesiva con 20 niveles y 159 hojas.

Conclusión. Este árbol es demasiado complejo para ser fiable. Es imprescindible limitar el crecimiento mediante parámetros de poda.

3.2. Efecto de la Profundidad Máxima

Analizamos cómo cambia el rendimiento del árbol al limitar su crecimiento con `max_depth`.

Cuadro 10: Efecto de la profundidad máxima

<code>max_depth</code>	Train Acc	CV Acc	Test Acc	Prof. real	N° Hojas
1	0.7893	0.7894	0.7765	1	2
2	0.8048	0.7922	0.7598	2	4
3	0.8287	0.8091	0.8045	3	8
5	0.8525	0.8077	0.7933	5	22
7	0.8834	0.7922	0.8045	7	38
10	0.9101	0.7740	0.7877	10	67
15	0.9705	0.7417	0.7933	15	135
None	0.9831	0.7417	0.8045	20	159

El punto óptimo está en `max_depth=3`. Con esta profundidad el modelo logra su mejor rendimiento en validación cruzada (0.8091) y mantiene un sólido 0.8045 en test. Es lo suficientemente complejo para captar los patrones importantes pero sigue siendo sencillo con solo 8 hojas.

Con valores de 1 o 2 el árbol es demasiado simple. A partir de profundidad 7 el overfitting crece rápidamente. La precisión en entrenamiento sube sin parar hasta 0.9831 mientras que la validación cruzada cae hasta 0.7417.

Conclusión. Limitar la profundidad a 3 niveles es la mejor estrategia. Nos da un clasificador muy fácil de interpretar que además es el que mejor generaliza.

3.3. Criterios de División

Comparamos los criterios de Gini, Entropía y Log Loss para comprobar si la métrica de división influye significativamente.

Cuadro 11: Comparación de criterios de división

Criterio	CV Accuracy	Test Accuracy	N° Hojas
Gini	0.7417	0.8045	159
Entropy	0.7403	0.7989	155
Log Loss	0.7403	0.7989	155

Las diferencias entre las tres opciones son prácticamente insignificantes. Gini genera ligeramente más hojas pero obtiene un test accuracy marginalmente superior. Nos quedamos con Gini por ese pequeño extra de precisión.

3.4. Poda Previa: Grid Search de Hiperparámetros

Realizamos una búsqueda exhaustiva con `GridSearchCV` para encontrar la combinación ideal de `max_depth`, `min_samples_split`, `min_samples_leaf` y `max_leaf_nodes`.

Cuadro 12: Grid Search de poda previa: mejores configuraciones

<code>max_depth</code>	<code>min_split</code>	<code>min_leaf</code>	<code>max_nodes</code>	CV Acc	Test Acc
5	10	1	20	0.8147	0.7989
5	10	1	50	0.8147	0.7989
5	20	10	20	0.8133	0.7709
5	10	10	50	0.8133	0.7709
5	10	1	None	0.8133	0.7989

La combinación ganadora utiliza `max_depth=5` junto a un límite de 20 nodos hoja. La precisión en validación cruzada ha subido hasta 0.8147 mientras que la brecha entre entrenamiento y prueba se ha reducido drásticamente, confirmando que el modelo ya no está memorizando ruido sino aprendiendo reglas reales.

3.5. Poda Posterior: Cost-Complexity Pruning

A diferencia de los métodos anteriores, aquí dejamos que el árbol se desarrolle por completo y después eliminamos las ramas menos informativas usando `ccp_alpha`.

Cuadro 13: Cost-Complexity Pruning: valores representativos

ccp_alpha	CV Acc	Test Acc	Prof.	N° Hojas
0.00000 (sin poda)	0.7417	0.8045	20	159
0.00086	0.7445	0.8045	18	118
0.00134	0.7585	0.8156	15	77
0.00218	0.7852	0.8045	11	25
0.00702	0.8077	0.7877	3	6
0.03556	0.7824	0.7765	1	2

A medida que aumentamos alpha, el número de hojas cae de forma drástica. El mejor CV accuracy (0.8077) se obtiene con `ccp_alpha=0.00702`, que produce un árbol muy compacto de solo 3 niveles y 6 hojas. Eliminar el ruido mediante la poda posterior mejora significativamente la generalización.

3.6. Importancia de Características

Utilizamos `feature_importances_` del mejor clasificador para cuantificar el peso de cada característica basándose en la impureza de Gini.

Cuadro 14: Importancia de características (Gini) en el árbol de decisión

Ranking	Feature	Importancia
1	Sex_male	0.5437
2	Pclass_3	0.1356
3	Age	0.1318
4	FamilySize	0.0690
5	Fare	0.0687
6	Pclass_2	0.0220
7	Embarked_S	0.0179
8	SibSp	0.0114
9	Embarked_Q	0.0000
10	Parch	0.0000

El sexo domina el modelo con una importancia de 0.5437 (más de la mitad del peso total). La tercera clase (0.1356) y la edad (0.1318) ocupan los siguientes puestos con impacto casi idéntico. `Embarked_Q` y `Parch` tienen importancia nula, el árbol los ignora por completo.

Conclusión. El modelo se apoya fundamentalmente en tres pilares, el sexo, la clase y la edad. El resto apenas aporta matices al resultado.

3.7. Visualización del Árbol

La Figura 2 muestra la estructura del mejor árbol limitando la visualización a 4 niveles de profundidad para asegurar la legibilidad. Es la mejor forma de comprobar visualmente cómo el sexo y la clase van segmentando el dataset hasta llegar a la predicción final.

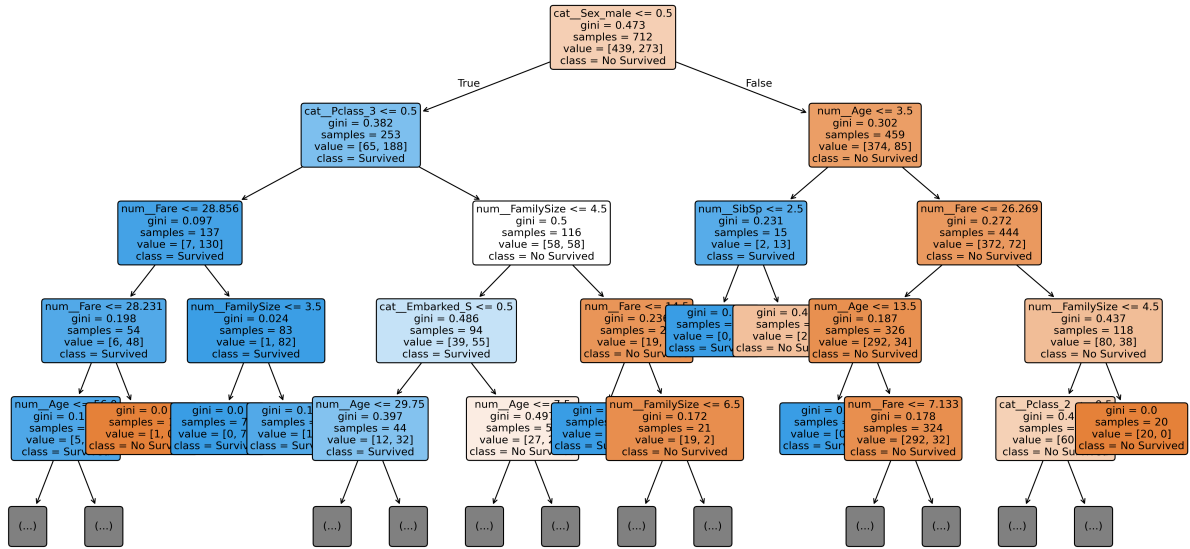


Figura 2: Visualización del árbol de decisión (max_depth=4)

3.8. Evaluación Final (Parte 2)

Cuadro 15: Resumen Parte 2: comparativa de árboles de decisión

Modelo	CV Acc	Test Acc	F1	Prof.	Nº Hojas
Sin restricciones	0.7417	0.8045	0.7445	20	159
Mejor poda previa	0.8147	0.7989	0.6897	5	20
Mejor poda posterior	0.8077	0.7877	0.7031	3	6

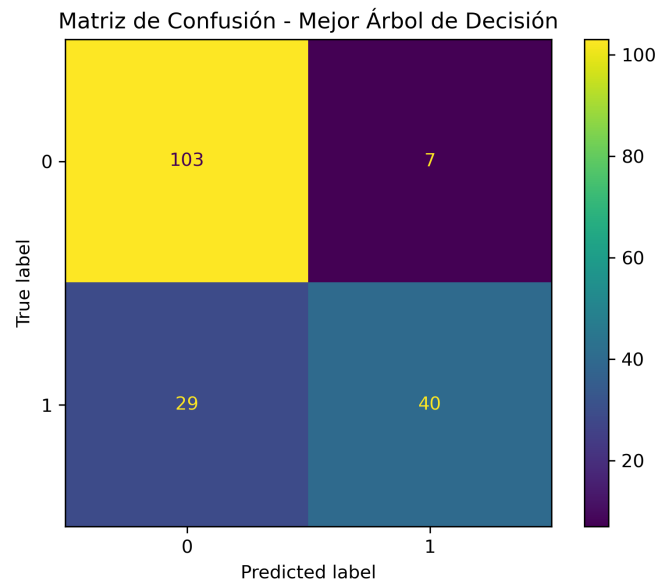


Figura 3: Matriz de confusión del mejor árbol de decisión

El mejor árbol alcanza un 80 % de precisión general en test, situándose a la par de los SVM. La Figura 3 muestra que es extremadamente eficaz detectando a los no supervivientes (recall de 0.94), pero solo identifica al 58 % de los supervivientes. La poda previa es la más equilibrada con un CV de 0.8147 y solo 20 hojas, mientras que la poda posterior simplifica al máximo con apenas 6 hojas. El modelo sin restricciones tiene un F1-Score ligeramente superior pero sus 159 hojas lo convierten en una estructura demasiado ruidosa.

Conclusión. No hace falta un árbol gigante para predecir bien. Un modelo podado con 20 hojas es mucho más robusto y fácil de interpretar.

4. Parte 3: Random Forest

4.1. Random Forest con Configuración por Defecto

En lugar de usar un único árbol entrenamos un conjunto de 100 árboles de decisión independientes para que voten el resultado final y así reducir la varianza. Una ventaja es el **OOB Score** que estima el acierto sin necesidad de un conjunto de validación externo.

Cuadro 16: Random Forest por defecto

Configuración	OOB Score	Test Acc	F1-Score
RF por defecto (n=100)	0.8048	0.7989	0.7273

El OOB Score de 0.8048 es prácticamente idéntico al 0.7989 del test, confirmando que la estimación interna es fiable y que el bosque generaliza correctamente. Con un F1-Score de 0.7273 ya supera los resultados de los árboles individuales optimizados.

4.2. Efecto del Número de Estimadores

Cuadro 17: Efecto de n_estimators en Random Forest

n_estimators	OOB Score	Test Acc	Tiempo (s)
10	0.7725	0.7821	0.038
25	0.8048	0.7989	0.071
50	0.7949	0.7989	0.128
100	0.8048	0.7989	0.220
200	0.8076	0.8156	0.447
500	0.8132	0.8156	1.075

Con solo 10 estimadores el modelo se queda corto (OOB=0.7725). Entre 25 y 100 árboles el rendimiento se mantiene estable. El salto de calidad definitivo llega a los 200 estimadores donde la precisión en test sube hasta 0.8156, manteniéndose con 500 árboles. Merece la pena usar al menos 200 estimadores.

4.3. Efecto de max_features

Cuadro 18: Efecto de max_features

max_features	OOB Score	Test Acc
sqrt	0.8048	0.7989
log2	0.8048	0.7989
0.5	0.8174	0.8045
None (todas)	0.8188	0.8156

Las opciones `sqrt` y `log2` devuelven los mismos valores porque en datasets pequeños ambas fórmulas dan un número de características muy similar. El mejor resultado aparece

al usar todas las características (None), con un OOB de 0.8188 y test de 0.8156. Para este problema concreto no necesitamos forzar tanto la aleatoriedad.

4.4. Grid Search

Los mejores parámetros encontrados son `n_estimators=200`, `max_features='sqrt'`, `max_depth=10`, `min_samples_leaf=2`, con un CV de **0.8287** y un test de **0.8045**.

Es interesante que `max_features=sqrt` resultó ser el mejor en la búsqueda conjunta. Aunque en las pruebas individuales usar todas las variables era mejor, la combinación con el resto de hiperparámetros demuestra que fomentar la diversidad entre los árboles es la clave para ganar precisión. El CV score de 0.8287 lo sitúa como uno de los modelos más potentes de toda la práctica.

4.5. Importancia de Características

Comparamos la importancia Gini con la importancia por permutación, que suele ser más fiable cuando existen variables correlacionadas.

Cuadro 19: Importancia: Gini vs. Permutación en Random Forest

R	Feature	Gini Imp.	Perm. Imp.
1	Sex_male	0.3469	0.1693
2	Fare	0.2254	0.0240
3	Age	0.1716	0.0453
4	Pclass_3	0.0857	0.0391
5	FamilySize	0.0596	0.0028
6	Embarked_S	0.0301	0.0050
7	SibSp	0.0290	-0.0022
8	Parch	0.0244	0.0078
9	Pclass_2	0.0197	-0.0056
10	Embarked_Q	0.0078	0.0011

El sexo sigue siendo el factor más decisivo en ambas listas. Sin embargo, Fare presenta una discrepancia curiosa. Gini le otorga 0.2254 pero la permutación lo baja a 0.0240, porque la importancia Gini tiende a inflar variables numéricas con muchos valores únicos. La permutación nos revela que una vez que conocemos el sexo, la clase y la edad, el resto de variables apenas mueven la aguja.

4.6. Extra Trees

El algoritmo Extra Trees elige umbrales de división de forma totalmente aleatoria. Entrenamos 100 árboles y obtenemos un CV de 0.7740 y un test accuracy de **0.8268**, el mejor resultado sobre el conjunto de prueba hasta ahora. Aunque su validación es algo más inestable, la simplificación radical de los umbrales ha funcionado especialmente bien con las muestras de nuestro test.

4.7. Evaluación Final (Parte 3)

Cuadro 20: Resumen Parte 3: comparativa de ensambles basados en árboles

Modelo	CV Acc	Test Acc	F1	OOB	t (s)
DT baseline	0.7417	0.8045	0.7445	N/A	0.000
RF por defecto	0.8035	0.7989	0.7273	0.8048	0.220
RF mejor config	0.8287	0.8045	0.7244	N/A	0.363
Extra Trees	0.7740	0.8268	0.7669	N/A	0.137

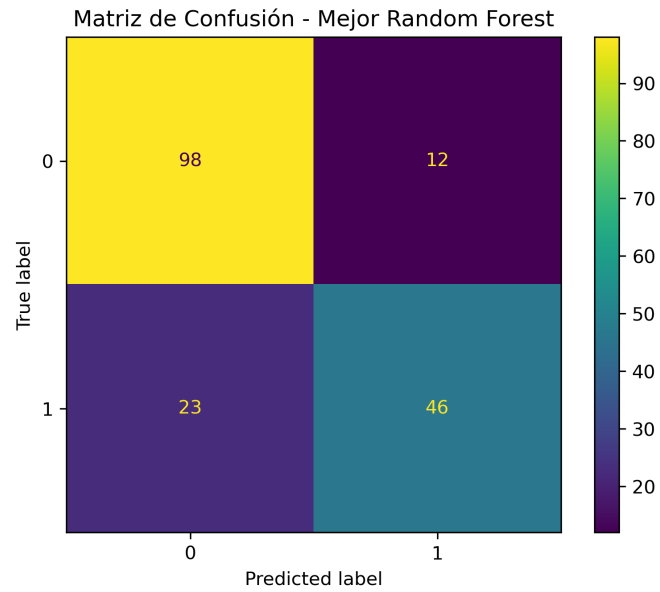


Figura 4: Matriz de confusión del Random Forest

El RF optimizado es el modelo más fiable y estable ($CV=0.8287$) mientras que Extra Trees se comporta como un “sprinter” con el mayor test accuracy (0.8268). La Figura 4 muestra un modelo equilibrado con $precision=0.79$ y $recall=0.67$ para los supervivientes.

5. Parte 4: Gradient Boosting

5.1. Gradient Boosting con Configuración por Defecto

El Gradient Boosting construye árboles de forma secuencial, donde cada nuevo árbol se fija en los fallos del anterior para corregirlos. Con parámetros por defecto obtiene un CV de **0.8260** y un test de **0.7877**. La diferencia entre ambos valores indica que el modelo tiene un problema de sobreajuste que habrá que ajustar.

5.2. Efecto de `n_estimators` y `Learning Rate`

Estos dos parámetros funcionan siempre en equipo, ya que velocidad baja necesita más árboles y viceversa.

Cuadro 21: Efecto de `n_estimators` y `learning rate`

<code>n.est</code>	<code>lr</code>	CV Acc	Test Acc
50	0.10	0.8245	0.7989
100	0.10	0.8260	0.7877
200	0.10	0.8218	0.7989
50	0.50	0.8105	0.8045
100	0.05	0.8260	0.7989
200	0.01	0.8217	0.8045

Con `lr=0.5`, solo 50 árboles consiguen un 0.8045 en test (aprendizaje rápido). Con `lr=0.01`, 200 árboles dan el mismo resultado (aprendizaje gradual y seguro). Curiosamente, la configuración de 100 árboles con `lr=0.1` saca la mejor nota en validación pero la peor en test, demostrando que a veces el modelo se confía demasiado.

Conclusión. Para este problema es mejor usar velocidad baja con muchos árboles, o velocidad alta con pocos. Ambas estrategias evitan la memorización.

5.3. Efecto de `max_depth`

En Gradient Boosting los árboles base son deliberadamente simples.

Cuadro 22: Efecto de `max_depth` en Gradient Boosting

<code>max_depth</code>	Train Acc	CV Acc	Test Acc
1 (stump)	0.8230	0.8119	0.8045
2	0.8694	0.8147	0.8101
3	0.9045	0.8260	0.7877
5	0.9593	0.7979	0.8045

Con profundidad 2 se alcanza el mejor resultado en test (0.8101). Al subir a profundidad 3 y 5, la precisión en entrenamiento aumenta hasta 0.9593 pero el acierto en test disminuye, confirmando el sobreajuste. El valor de 2 es el parámetro más equilibrado.

5.4. Grid Search

Los mejores parámetros son `learning_rate=0.1`, `max_depth=3`, `n_estimators=100`, `subsample=1.0`, con CV de **0.8260** y Test de **0.7877**. Es la misma configuración por defecto, indicando que el Gradient Boosting ya estaba bien calibrado de serie. El hecho de que `subsample` se quede en 1.0 indica que el modelo prefiere usar el total de los datos. A pesar de ser un algoritmo avanzado, no ha logrado superar al Random Forest.

5.5. AdaBoost

AdaBoost utiliza una estrategia basada en pesos. Cuando un árbol falla, el siguiente se esfuerza más en ese caso concreto.

Cuadro 23: AdaBoost: exploración de hiperparámetros

n_est	lr	CV Acc	Test Acc
50	1.0	0.8063	0.7933
100	1.0	0.8063	0.7765
100	0.5	0.8077	0.7877
200	0.1	0.8091	0.7933

Al subir de 50 a 100 árboles con `lr=1.0`, la precisión en test baja de 0.7933 a 0.7765, demostrando que más estimadores sin reducir la velocidad puede perjudicar. La mejor puntuación en CV aparece con 200 árboles y `lr=0.1` (0.8091). AdaBoost es sólido pero no llega a superar al Gradient Boosting o al Random Forest.

5.6. Evaluación Final (Parte 4)

Cuadro 24: Resumen Parte 4: comparativa de boosting

Modelo	CV Acc	Test Acc	F1	t (s)
RF mejor config	0.8287	0.8045	0.7244	0.359
GB defecto	0.8260	0.7877	0.6885	0.134
GB mejor config	0.8260	0.7877	0.6885	0.135
AdaBoost	0.8063	0.7765	0.7183	0.178

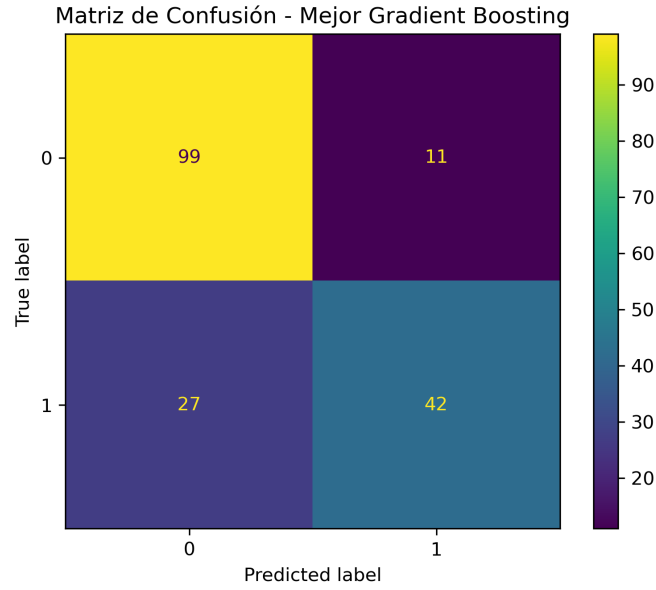


Figura 5: Matriz de confusión del Gradient Boosting

El Random Forest sigue siendo el modelo más sólido (Test=0.8045, CV=0.8287). La Figura 5 confirma que ambas versiones del Gradient Boosting obtienen exactamente el mismo resultado (0.7877 en test), indicando que los parámetros de serie ya eran los más adecuados. AdaBoost es el modelo con menor precisión en test (0.7765) aunque su F1-Score (0.7183) es mejor que el del GB.

Conclusión. Tras probar todas las técnicas de boosting, el Random Forest se mantiene como la solución más equilibrada y fiable.

6. Comparación y Evaluación Global

Reunimos las mejores configuraciones de cada uno de los modelos analizados a lo largo de la práctica para una comparación directa.

Cuadro 25: Comparativa final: mejor configuración de cada método

Modelo	CV Accuracy	Test Acc	Prec.	Recall	F1	t (s)
SVM (RBF)	0.8385 ± 0.021	0.8045	0.8400	0.6087	0.7059	0.015
Decision Tree	0.8147 ± 0.038	0.7989	0.8511	0.5797	0.6897	0.003
Random Forest	0.8287 ± 0.027	0.8045	0.7931	0.6667	0.7244	0.362
Gradient Boosting	0.8260 ± 0.037	0.7877	0.7925	0.6087	0.6885	0.134
AdaBoost	0.8063 ± 0.032	0.7765	0.6986	0.7391	0.7183	0.175

6.1. Análisis de la Tabla Comparativa

- **El modelo con mejor desempeño.** El SVM (mejor config) se posiciona como el mejor modelo global. Logra la validación cruzada más alta (0.8385) y una precisión en el test de 0.8045. Aunque el Random Forest alcanza la misma precisión en el test, el SVM es un modelo más sencillo y mucho más rápido de entrenar.

- **Precisión y F1-Score.** El Random Forest obtiene el mejor F1-Score (0.7244), indicando un equilibrio superior entre precisión y recall. Sin embargo, el SVM destaca por tener la Precisión más alta (0.8400) al predecir supervivientes, cometiendo muy pocos errores de falsos positivos.
- **Modelos de Boosting.** Tanto el Gradient Boosting como el AdaBoost se quedan ligeramente por detrás. A pesar de ser algoritmos más complejos, no han conseguido superar la barrera del 80 % en este dataset específico.

6.2. Evaluación del Modelo Ganador: SVM

La Figura 6 muestra la matriz de confusión del modelo ganador.

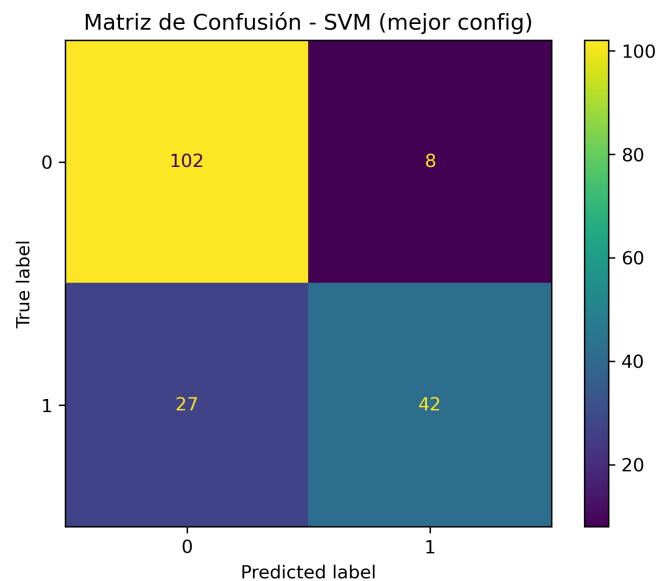


Figura 6: Matriz de confusión del mejor modelo global (SVM RBF, $C=1$, $\gamma=0.1$)

La matriz de confusión del SVM confirma su solidez.

- **Clase 0 (No supervivientes).** Acierta con 102 pasajeros y solo falla en 8 casos. Es excelente para identificar quiénes no sobrevivieron.
- **Clase 1 (Supervivientes).** Identifica correctamente a 42 pasajeros. Aunque deja de detectar a 27 (recall de 0.61), cuando el modelo predice que alguien sobrevive, acierta el 84 % de las veces.

7. Conclusiones

Para este problema del Titanic, el **SVM con núcleo RBF** es la opción más recomendable. Ofrece el mejor equilibrio entre capacidad de generalización (CV Accuracy de 0.8385), acierto real sobre datos nuevos (Test Accuracy de 0.8045) y eficiencia técnica, ya que su tiempo de ejecución es prácticamente instantáneo (0.015 segundos).

1. **SVM.** El kernel RBF supera a los demás kernels. El escalado es crítico. Los valores moderados de C y γ evitan el overfitting.

2. **Árboles de Decisión.** Sin restricciones sobreajustan gravemente (train=98.3 %, test=80.5 %). La poda previa y posterior reducen la complejidad sin perder rendimiento.
3. **Random Forest.** Mejora la estabilidad respecto al árbol individual. El Grid Search alcanza un CV de 0.8287 con 200 estimadores. Extra Trees sorprende con el mayor test accuracy aislado (0.8268).
4. **Gradient Boosting y AdaBoost.** A pesar de su sofisticación, no superan al RF ni al SVM en este dataset. La relación inversa entre learning rate y número de estimadores se confirma experimentalmente.
5. **Variables clave.** En todos los algoritmos, el sexo, la clase del billete y la edad son los predictores más importantes, coherente con los hechos históricos del Titanic.